B.M.S. COLLEGE OF ENGINEERING, BENGALURU

(Autonomous Institute, Affiliated to VTU)

Department of Computer Science and Engineering

SOFT COMPUTING (23CS7PESCN / 23CS8PESCN)

Alternative Assessment Tool (AAT) Report

GA-Optimised Fuzzy Traffic-Signal Control for a Congested Three-Junction Corridor

Problem Statement 12: Optimise a fuzzy traffic-signal controller for a congested three-junction corridor to reduce average vehicle delay compared with fixed-time plans.

Technique: Fuzzy logic + Evolutionary Algorithm (Units 2 and 3) | SDG 11

Submitted by

Name	USN
<Student name 1>	<USN>
<Student name 2>	<USN>
<Student name 3>	<USN>

Faculty in charge: <Faculty in charge>

Academic Year 2026-2027

Abstract

Fixed-time signal plans cannot react when corridor traffic rises, falls or shifts direction. This work formulates the design of an adaptive controller for a congested three-junction arterial as an optimisation problem: choose the membership functions, decision threshold and green-time limits of a Mamdani fuzzy controller to minimise average delay per vehicle. Each junction runs a nine-rule fuzzy controller that weighs the demand on green, including vehicles about to arrive from upstream, against the queue on red. An 11-gene real-coded genetic algorithm (tournament selection, simulated binary crossover, Gaussian mutation, elitism) tunes it inside a purpose-built corridor simulator. All three are implemented from scratch in Python without library functions. On ten unseen seeds, the GA-tuned controller cut delay by up to 25.4 % compared with a Webster fixed-time plan with green wave, but was 9.9 % worse in light traffic because it had been tuned on a peak hour. Retrained on light, moderate and peak demand, it reduced delay significantly in all six scenarios, by 3.8 % to 26.3 %.

Keywords: fuzzy logic control, Mamdani inference, genetic algorithm, traffic signal, arterial corridor, vehicle delay, SDG 11.

1. Introduction

Signalised junctions decide how much of every hour each stream of traffic may use the road. In most Indian cities the signals on an arterial road are operated by fixed-time plans: a cycle length and a green split that are computed once from traffic counts and then repeated all day, or for a few time-of-day periods. Traffic, however, is not constant. Demand builds up and decays around the peak hour, the dominant direction reverses between the morning and the evening, and side streets surge when schools, offices or markets empty. A fixed plan that is right for the average is wrong for most individual cycles: green time is wasted on empty approaches while queues build on the others.

The people who face this problem are commuters and bus passengers, who lose time, and the city, which pays for it in fuel, emissions and road space. The problem is sharper on a corridor than at an isolated junction, because the junctions interact. Vehicles released by one junction arrive as a platoon at the next one 25-30 s later. If the downstream signal is red, the platoon stops, and if the queue grows long enough it can block the upstream junction (spill-back).

Fuzzy logic is a natural fit for this decision. An experienced traffic constable does not compute optimal cycle lengths. They reason with rules such as "if the queue on red is long and few vehicles are still coming on green, end the green". Pappis and Mamdani [3] showed in 1977 that a fuzzy controller built on such rules can outperform a vehicle-actuated controller at an isolated junction, and many later controllers followed [4], [5], [19]. The weakness of a fuzzy controller is that its performance depends on where the fuzzy sets are placed (what counts as a "long" queue), and choosing these by hand is guesswork. An evolutionary algorithm can tune them automatically against a simulation of the actual corridor.

Objective. The objective of this work is to design, implement from first principles and evaluate a fuzzy traffic-signal controller for a congested three-junction corridor, to tune it with an evolutionary algorithm so that average vehicle delay is minimised, and to measure the resulting delay against fixed-time plans under several demand patterns. The specific goals are:

- Formulate controller design as a bounded, simulation-based optimisation problem (Chapter 2).
- Implement a Mamdani fuzzy inference system and a real-coded genetic algorithm without library functions, together with a corridor traffic simulator (Chapters 5 and 6).
- Compare the GA-tuned fuzzy controller with a Webster fixed-time plan, a GA-tuned fixed-time plan and the untuned fuzzy controller on random seeds not used in training (Chapter 7).
- Analyse the effect of key parameters and state the limits of the approach (Chapter 8).

2. Problem Formulation

2.1 The corridor

The study corridor (Figure 2.1) is a stylised east-west urban arterial with three signalised junctions J1, J2 and J3, spaced 350 m and 400 m apart (free-flow travel times 25 s and 29 s at 50 km/h). The arterial has two lanes in each direction and each cross street has one lane in each direction. Every junction runs two phases: phase A serves the arterial (eastbound and westbound) and phase B serves the side street (northbound and southbound). Of the vehicles leaving an arterial approach, 85 % continue to the next junction and the rest turn off. Of the vehicles leaving a side street, 15 % turn onto the arterial in each direction.

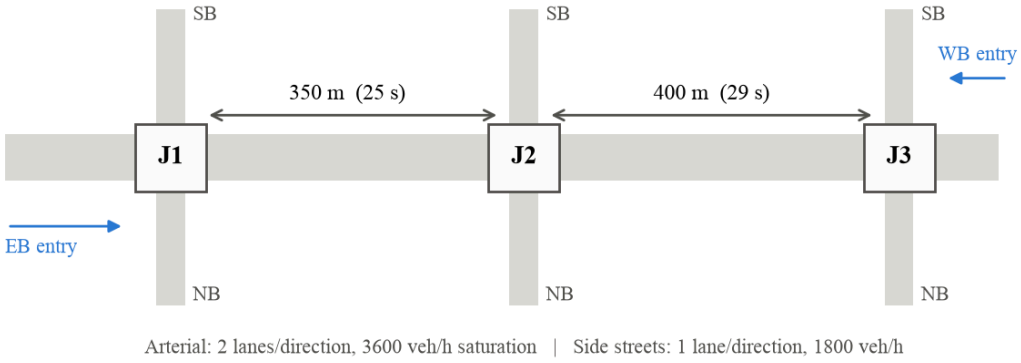


Figure 2.1: Layout of the three-junction study corridor.

2.2 Decision variables

All three junctions use the same controller design (the same rule base and parameters), so the design is described by an 11-dimensional real vector x, listed in Table 2.1. The break points of each input's fuzzy sets are encoded as a start value and two positive increments. This guarantees a < b < c for every chromosome, so ordering constraints never need repair.

Table 2.1: Decision variables (chromosome) of the fuzzy controller.

Gene	Meaning	Bounds	Hand-tuned value
x1	Green demand: end of 'Low' plateau (a)	[0, 6]	2
x2	Green demand: b - a	[0.5, 7]	4
x3	Green demand: c - b	[0.5, 7]	6
x4	Red queue: end of 'Low' plateau (a)	[0, 8]	3
x5	Red queue: b - a	[0.5, 8.5]	5
x6	Red queue: c - b	[0.5, 8.5]	7
x7	Switch threshold theta on the urgency output	[0.3, 0.8]	0.5
x8	Arterial minimum green (s)	[7, 20]	10
x9	Arterial maximum green (s)	[25, 90]	60
x10	Side-street minimum green (s)	[7, 15]	7
x11	Side-street maximum green (s)	[15, 60]	40

2.3 Objective function

For a scenario s and a random seed k, the simulator generates the same set of vehicles V(s, k) for every controller. The delay d_v of vehicle v is the total time it spends waiting in stop-line queues along its route. The objective is the average delay per vehicle, averaged over the training seeds K:

$$\text{minimise } J(x) = (1/|K|)\ \Sigma_k\ [\ (1/|V(s,k)|)\ \Sigma_v\ d_v(x, s, k)\ ] \tag{2.1}$$

Vehicles still queued at the end of the hour are charged the time they have already waited. Without this rule a controller could look good by never serving a side street. The training scenario is the one-hour peak build-up and decay (s = peak_build) with K = {1001, 1002, 1003}.

2.4 Constraints

- Bound constraints: $x_{i,\min} \le x_i \le x_{i,\max}$ for every gene (Table 2.1).
- Ordering of fuzzy break points $a < b < c$: satisfied by construction (increment encoding).
- Minimum green ≥ 7 s and minimum green < maximum green: guaranteed by the bounds.
- Safety inter-green of 3 s amber + 1 s all-red between conflicting greens: enforced by the controller state machine and not tunable.
- Capacity: an approach discharges at most its saturation flow (1800 veh/h/lane) during effective green, which starts 2 s after the green begins.
- Storage: a vehicle cannot enter a link whose stored vehicles reach 100 (J1-J2) or 114 (J2-J3) cars (spill-back blocks the approach).

2.5 Assumptions

- Vehicles arrive at the corridor entries as Poisson processes whose rate may vary with time.
- Queues are vertical (point) queues at the stop line. Links are traversed at free-flow speed.
- All vehicles are passenger-car units. Pedestrian phases, buses and two-wheeler behaviour are not modelled.
- Detectors report each approach's queue and the vehicles that will arrive within the next 7 s (an advance detector about 100 m upstream).
- Two-phase operation, with turning traffic served in the same phase as its approach.

The fixed-time baseline can be written as an optimisation problem in the same way. The GA-tuned fixed-time plan uses six genes: a common cycle length C in [40, 150] s, the arterial

share of effective green at each junction in [0.3, 0.85], and the offsets of J2 and J3 as fractions of C. It is trained with exactly the same GA, budget and fitness as the fuzzy controller, which makes the comparison fair.

3. Technique Selection

The problem statement suggests fuzzy logic combined with an evolutionary algorithm. The fuzzy inference system is taken from Unit 2 (fuzzy sets [1], membership functions, fuzzy rules, Mamdani inference [2] and defuzzification) and the genetic algorithm from Unit 3 (evolutionary computation [8], [9], GA operators and real-coded GAs). Both are covered in the course texts [10], [17]. The two play different roles: fuzzy logic is the controller that runs every second at every junction, and the GA is the offline optimiser that designs it.

3.1 Why fuzzy logic for the controller

- Traffic states are naturally linguistic and imprecise: queue counts from cameras or loops are noisy, and "long queue" has no sharp boundary.
- The rule base is small (nine rules) and readable by traffic engineers, so the controller can be inspected and trusted. A neural network controller would be a black box.
- No training data of 'correct' signal decisions exist, so supervised methods (neural networks, ANFIS) have nothing to learn from. A fuzzy controller can be written from expert knowledge and then tuned.
- Inference is cheap enough to run on a signal controller every second (Section 7.7).

3.2 Why an evolutionary algorithm for tuning

Each evaluation of J(x) is a stochastic one-hour simulation. The function has no gradient, is not smooth (a small change in a break point flips individual switching decisions) and is noisy. These properties rule out gradient-based methods. Exhaustive search is also infeasible: only ten levels per gene would need 10^11 simulations. A population-based evolutionary algorithm needs only fitness values, tolerates noise and explores many regions of the space at once.

Table 3.1: Alternatives considered for tuning the controller.

Alternative	Assessment for this problem
Gradient descent / Newton	Not applicable: J(x) comes from a simulation and has no derivative.
Grid or random search	Grid is infeasible in 11-D. Random search is used as the GA's first population, and the GA improves on it substantially (Figure 7.3).
Binary-coded GA	Works, but precision is tied to string length and Hamming cliffs hinder fine tuning of continuous break points [7].
Evolution strategies / EP	Also suitable for real vectors; GA chosen because crossover can combine a good green-demand partition from one parent with a good red-queue partition from another.
ANFIS / neural control	Needs input-output training pairs of optimal decisions, which do not exist.
Real-coded GA (chosen)	Real genes, SBX crossover with tunable spread [7], simple to implement without libraries, well-documented for fuzzy controller tuning [5].

4. Data Collection and Pre-processing

4.1 Sources of parameters

No detector counts of a real corridor were available to the team, so the study uses synthetic but realistic benchmark instances built from standard design values. Base saturation flow is taken as 1800 vehicles per hour per lane, the base value used by the Highway Capacity Manual [11]. Signal timing elements (amber 3 s, all-red clearance, minimum green) follow the usual practice described in IRC:93 [12] and Webster [6]. Start-up lost time is 2 s per green, so each phase loses 6 s and a two-phase cycle loses L = 12 s. Queued vehicles occupy 7 m each. These values are summarised in Table 4.1.

Table 4.1: Corridor and signal parameters.

Parameter	Value
Arterial lanes / side-street lanes (per direction)	2 / 1
Saturation flow	1800 veh/h/lane (1 veh/s on the arterial, 0.5 veh/s on a side street)
Link lengths and travel times	350 m / 25 s, 400 m / 29 s
Link storage (2 lanes, 7 m per car)	100 and 114 vehicles
Amber + all-red, start-up lost time	3 s + 1 s, 2 s
Minimum green (safety floor)	7 s
Turning: arterial through / side to arterial	0.85 / 0.15 each way
Detection horizon (advance detector)	7 s
Simulation length and time step	3600 s, 1 s

4.2 Demand scenarios

Six one-hour demand scenarios were defined (Table 4.2). They cover steady light, moderate and congested traffic, plus three time-varying patterns that a fixed plan cannot follow. Flows are entry flows in vehicles per hour. Arrivals are generated each second as Poisson counts with the instantaneous rate (Knuth's method [14]). Each vehicle also receives its turning

random numbers at generation time, so every controller faces exactly the same vehicles for a given seed (common random numbers). This removes most of the noise from paired comparisons.

Table 4.2: Demand scenarios (veh/h) and the resulting Webster cycle length.

Scenario	EB / WB entry	Side NB/SB at J1, J2, J3	Time profile	Webster C
Off-peak (light, steady)	900 / 800	180/160, 240/220, 170/150	flat	40 s
Moderate (steady)	1500 / 1300	330/300, 420/390, 320/280	flat	60 s
Peak (congested, steady)	1900 / 1650	400/370, 500/470, 390/350	flat	95 s
Peak build-up and decay (training)	1500 / 1300	330/300, 420/390, 320/280	all flows x0.85 -> x1.30 (20-40 min) -> x0.85	78 s
Tidal shift EB to WB	1500 / 1300	330/300, 420/390, 320/280	EB x1.30 -> x0.70, WB x0.70 -> x1.45 over the hour	60 s
Side-street surge at J3	1500 / 1300	330/300, 420/390, 320/280	J3 side streets x2.0 from 21 to 41 min	60 s

4.3 Pre-processing and normalisation

- **Flow propagation.** Mean entry flows are propagated through the turning fractions to obtain the flow arriving at every approach. These flows give the critical flow ratios y used by Webster's method (Section 5.6).
- **Per-lane normalisation of controller inputs.** Queues are divided by the number of lanes, so 10 cars on the 2-lane arterial (5 per lane) and 5 cars on a 1-lane side street are comparable.
- **Clamping.** Inputs are clamped to their universes, [0, 20] veh/lane for green demand and [0, 25] veh/lane for red queue. Larger values are treated as fully 'High'.
- **Discretisation.** Inputs are multiples of 0.5 veh/lane, so each controller caches its inference results. A cached result is identical to recomputing it.
- **Training/test split.** The GA sees only the peak build-up scenario with seeds 1001-1003. All reported results use seeds 1-10 on all six scenarios, five of which the GA never saw.

4.4 Simulator validation

Before using the simulator as a fitness function it was validated against Webster's delay formula for an isolated fixed-time approach with random arrivals [6]. Only the J1 eastbound approach is loaded, and its average delay over 8 seeds of two hours is compared with the formula (Table 4.3). Agreement is within a few per cent up to a degree of saturation of about 0.85. This is the range where Webster's formula itself is considered reliable.

Table 4.3: Simulated delay versus Webster's formula (isolated approach, s = 3600 veh/h).

q (veh/h)	C (s)	g (s)	x	Webster d (s)	Simulated d (s)	Difference
800	60	30	0.44	10.26	10.02	-2.4 %
1200	60	30	0.67	12.40	11.78	-5.0 %
1500	80	40	0.83	19.93	19.06	-4.4 %
1700	90	50	0.85	19.73	18.79	-4.8 %

5. Algorithm Design

5.1 Fuzzy controller: inputs and membership functions

At every second after the minimum green, the controller at each junction computes two crisp inputs:

$$G = \max \text{ over green approaches of } (\text{queue} + \text{arrivals in next 7 s}) / \text{lanes} \tag{5.1}$$

$$R = \max \text{ over red approaches of queue / lanes} \tag{5.2}$$

Including the vehicles about to arrive lets the downstream junction keep the green for a platoon released upstream. This is how the junctions coordinate without a common cycle. Each input is covered by three sets, Low, Medium and High, defined by break points $a < b < c$. Low is a left shoulder (1 up to a, 0 from b), Medium is a triangle (a, b, c) and High is a right shoulder (0 up to b, 1 from c):

$$\mu_Low(x) = \max(0, \min(1, (b - x)/(b - a))), \mu_Med(x) = \max(0, \min((x - a)/(b - a), (c - x)/(c - b))) \tag{5.3}$$

$$\mu_High(x) = \max(0, \min(1, (x - b)/(c - b))) \tag{5.4}$$

The memberships of the three sets always add up to one (a strong fuzzy partition, verified by a unit test). The output variable 'switch urgency' u on [0, 1] has three fixed triangular sets: Low (0, 0, 0.5), Medium (0.25, 0.5, 0.75) and High (0.5, 1, 1).

5.2 Rule base and inference

Table 5.1: Rule base: urgency to end the current green (9 rules).

Green demand \ Red queue	Low	Medium	High
Low	Low	High	High
Medium	Low	Medium	High
High	Low	Low	Medium

Each rule reads, for example, "IF green demand is Low AND red queue is High THEN urgency is High". Inference is classical Mamdani [2]. The rule strength is the minimum of its two antecedent memberships. Each output set is clipped at the strongest rule that points to it

(min implication). The clipped sets are combined by the maximum (aggregation), and the crisp urgency is the centroid of the aggregated set, computed on 101 points:

$$u^* = \Sigma_k y_k \mu_{agg}(y_k) / \Sigma_k \mu_{agg}(y_k) \quad (5.5)$$

Decision. The green ends when u* > θ or when the maximum green is reached. It is never ended before the minimum green. Ending a green starts the 4 s inter-green, after which the other phase turns green. The rule base is kept fixed as expert knowledge, and the GA tunes the membership functions, θ and the green limits. Figure 5.1 shows the decision flow.

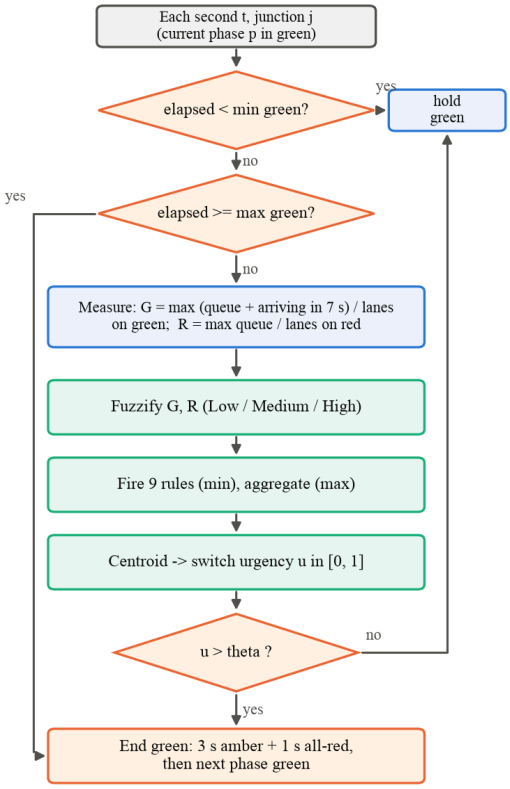


Figure 5.1: Flowchart of the per-second fuzzy signal decision.

5.3 Solution representation

A chromosome is the real vector x of Table 2.1. Decoding turns it into break points (cumulative sums of increments), the threshold θ and four green limits. The hand-tuned controller (last column of Table 2.1) is placed in the initial population so that expert knowledge is not lost. The remaining individuals are uniform random within the bounds.

5.4 Fitness function

fitness(x) = J(x) from equation (2.1), the mean of the average delay over three one-hour simulations of the training scenario. Lower is better. Because the seeds are fixed, the fitness is deterministic for a given x, so elites are not re-evaluated.

5.5 Genetic operators and parameters

- **Selection:** tournament of size k = 3. Pick three individuals at random and keep the fittest.
- **Crossover:** simulated binary crossover (SBX) [7], applied with probability p_c = 0.9. Each gene is crossed with probability 0.5. With u uniform on [0, 1), the spread factor is β = (2u)^(1/(η+1)) if u <= 0.5, else (1/(2(1−u)))^(1/(η+1)), with η_c = 15. The children are c1 = ½[(1+β)p1 + (1−β)p2] and c2 = ½[(1−β)p1 + (1+β)p2], clamped to the bounds.
- **Mutation:** Gaussian, each gene with probability p_m = 1/n = 1/11. x_i ← x_i + σ (x_i,max − x_i,min) z, where z ~ N(0, 1) is generated by the Box-Muller transform [13] and σ decreases linearly from 0.15 to 0.03 over the run (exploration first, fine tuning later).
- **Replacement:** generational with elitism. The best 2 individuals are copied unchanged.
- **Termination:** a fixed budget of G = 40 generations of N = 30 individuals (1150 fitness evaluations, i.e. 3450 one-hour simulations per run).

Pseudocode of the GA:

```
P <- {hand-tuned genome} U {N-1 uniform random genomes}
F <- evaluate(P)                        # 3 simulations each, in parallel
for g = 0 .. G-1:
    sigma <- 0.15 + (0.03 - 0.15) * g/(G-1)
    P' <- the 2 best of P (elites)
    while |P'| < N:
        p1 <- tournament(P, F, k=3);  p2 <- tournament(P, F, k=3)
        (c1, c2) <- SBX(p1, p2, pc=0.9, eta=15)
        P' <- P' + {gauss_mutate(c1, sigma), gauss_mutate(c2, sigma)}
    F' <- elite fitness + evaluate(new children)
    P, F <- P', F'
return argmin_x F
```

Pseudocode of the controller at junction j, second t:

```
if in_green:
    if elapsed >= min_green[phase]:
        if elapsed >= max_green[phase]:  end_green()
        else:
            G <- max_a (queue[a] + arriving_within_7s[a]) / lanes[a]   over green
approaches
            R <- max_a  queue[a] / lanes[a]                           over red
approaches
            u <- centroid(max-aggregate(min-clip(rules, fuzzify(G), fuzzify(R))))
            if u > theta:  end_green()          # 3 s amber + 1 s all-red, then
other phase
    elapsed <- elapsed + 1
```

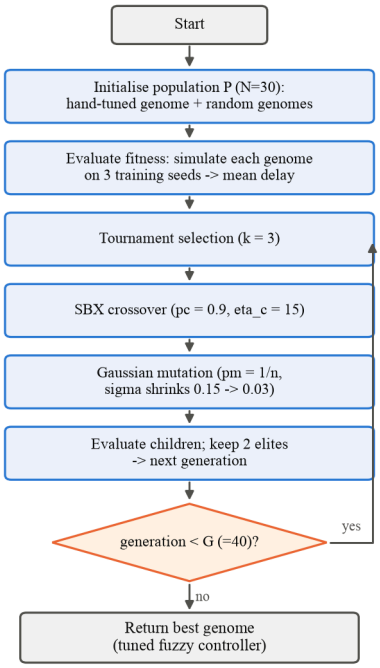


Figure 5.2: Flowchart of the real-coded genetic algorithm.

5.6 Baseline: Webster fixed-time plan with green wave

The baseline is the plan a traffic engineer would compute by hand. For each junction the critical flow ratios of the arterial and side-street phases are y_A = max(q_EB, q_WB)/s_A and y_B = max(q_NB, q_SB)/s_B, with Y = y_A + y_B. Webster's optimum cycle [6] is:

$$C_0 = (1.5\,L + 5) / (1 - Y) \tag{5.6}$$

The common cycle is the largest C_0 of the three junctions, clamped to [40, 150] s. Effective green C − L is split in proportion to y_A : y_B (equisaturation). Offsets of 0, 25 and 54 s (the link travel times) create an eastbound green wave, the idea that coordination tools such as TRANSYT [16] optimise. The plan is recomputed for each scenario from that scenario's own mean flows, so it has advance knowledge of the average demand that the fuzzy controller does not have.

6. Implementation

The project is written in Python 3. **The fuzzy inference system, the genetic algorithm and the traffic simulator are implemented without library functions.** No NumPy, SciPy, scikit-fuzzy, DEAP, PyGAD or other numerical, fuzzy or evolutionary library is used. The only facilities taken from the Python standard library are the uniform pseudo-random generator random.Random.random() (Mersenne Twister [18]), elementary functions from math (exp, log, sqrt, sin, cos), collections.deque for FIFO queues, json for saving results, and multiprocessing for evaluating a population in parallel on the CPU cores. Normal and Poisson

random numbers are generated by the project's own code. Matplotlib is used only to draw the figures in this report, after all results were computed.

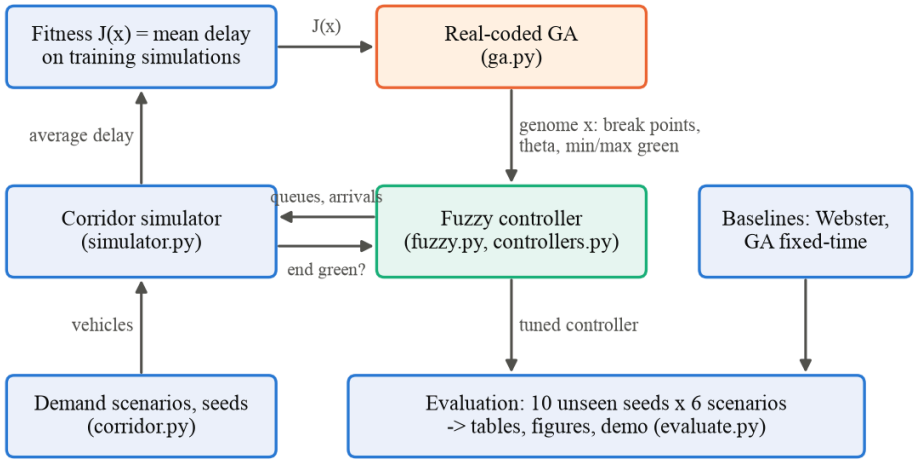


Figure 6.1: Program structure and data flow.

Table 6.1: Program modules.

Module	Lines	Responsibility
rng.py	50	Box-Muller normal and Knuth Poisson samplers on a uniform generator
fuzzy.py	159	Membership functions, linguistic variables, Mamdani FIS, signal rule base
ga.py	123	Real-coded GA: tournament, SBX, Gaussian mutation, elitism, history
corridor.py	129	Geometry, timing constants, scenarios, flow propagation
simulator.py	226	1-s corridor simulation, sensors, delay statistics
controllers.py	208	Webster plan, fixed-time decoder, fuzzy controller and chromosome
optimise.py	92	Runs independent GA runs with parallel fitness evaluation
evaluate.py	258	Test-set evaluation, paired t-test, sweeps, traces
plots.py	399	Report figures only (matplotlib)
build_demo.py	356	Offline HTML demo: corridor replay and interactive fuzzy controller
tests/ (3 files)	384	35 unit tests of the fuzzy system, GA and simulator

Main classes. MamdaniFIS stores the input variables, the rule list and the sampled output sets. Its infer() method performs fuzzification, rule evaluation, aggregation and centroid defuzzification. RealCodedGA.run() takes a batch evaluation function, so the same GA optimises both the fuzzy controller and the fixed-time plan. CorridorSim.run() advances the corridor one second at a time: arrivals, link arrivals, signal decisions and discharge, then statistics. Controllers share one interface, signal(t, j, sim) -> (green phase, seconds since green), so the simulator does not know which controller it is running.

Running the project. python3 -m unittest discover runs the 35 tests in about 6 s. optimise.py performs 5 independent GA runs for each design in about 2 minutes per run on an 8-core laptop. evaluate.py produces every table and figure input, and build_demo.py writes an offline

HTML demonstration (corridor replay, interactive fuzzy controller and results) used in the presentation.

7. Testing and Results

7.1 Functional testing

The fuzzy system, the GA and the simulator were each tested separately, without the others (Table 7.1). The test suite was re-run while this report was generated: 35 of 35 tests passed. Two of the fuzzy tests are property tests. They check that the urgency rises with the red queue and falls with the green demand. Classical Mamdani inference with max aggregation is only approximately monotone. Where an input crosses from one set to the next, two rules with the same consequent both fire at about 0.5, and their maximum dips slightly. The measured worst-case dip is 0.055 on the [0, 1] urgency scale, so the tests allow 0.06. This is a known property of the method, not a coding error.

Table 7.1: Unit test results (generated automatically while building this report).

#	Test case	Expected	Actual	Status
1	Centroid of one symmetric output set	0.5	as expected	Pass
2	Empty green & long red queue; busy green & empty red	u > 0.75; u < 0.25	as expected	Pass
3	Urgency vs green demand (property test)	non-increasing within 0.06	as expected	Pass
4	Urgency vs red queue (property test)	non-decreasing within 0.06	as expected	Pass
5	Crisp output for all 546 grid inputs	inside [0, 1]	as expected	Pass
6	Rule firing strength = min of antecedents	0.5, 0.5, 0	as expected	Pass
7	Triangle with peak on its edge (output sets)	membership 1 at the edge	as expected	Pass
8	Inputs outside the universe	clamped to the end sets	as expected	Pass
9	Left/right shoulder MFs	1 / 0.5 / 0 at the break points	as expected	Pass
10	Low+Medium+High memberships over the universe	sum = 1 at every point	as expected	Pass
11	Triangle MF at feet, peak, half-way	0, 1, 0.5	as expected	Pass
12	GA on 5-D sphere function	f < 0.01, x close to optimum	as expected	Pass
13	Best fitness over generations	never increases	as expected	Pass
14	GA on 4-D Rastrigin (multimodal)	f < 2.5	as expected	Pass
15	GA reproducibility	identical result for same seed	as expected	Pass
16	Seeded optimum survives (elitism)	fitness 0 returned	as expected	Pass
17	Gaussian mutation (500 mutations)	inside the gene bounds	as expected	Pass
18	SBX children (500 crossovers)	inside the gene bounds	as expected	Pass
19	SBX property	child mean = parent mean	as expected	Pass
20	Box-Muller normal samples (40 000)	mean 0, variance 1	as expected	Pass
21	Knuth Poisson samples, lambda = 0.1, 0.5, 2	mean = variance = lambda	as expected	Pass
22	Integer sampler range	all of 0..4 produced	as expected	Pass
23	Same seed, different controllers	same vehicles generated	as expected	Pass
24	3000 veh/h offered to 2 lanes, permanent green	served <= 1 veh/s	as expected	Pass
25	Downstream junction starved of green (spill-back)	link occupancy <= storage	as expected	Pass
26	1000 veh/h on 1 lane with 12 s green per 60 s	residual queue > 100	as expected	Pass

27	Side street held on red all hour	0 vehicles served	as expected	Pass
28	Same seed, same controller	identical results	as expected	Pass
29	Vehicles in = exited + queued + in transit	exact equality	as expected	Pass
30	Empty network	0 vehicles, 0 delay	as expected	Pass
31	Fixed-time 60 s plan with offsets	30 s / 8 s / 22 s; offset 10 s	as expected	Pass
32	Fuzzy controller timing over 1 hour	min <= green <= max, inter-green 4 s	as expected	Pass
33	Webster plans for all 6 scenarios	40 <= C <= 150, greens >= 7 s	as expected	Pass
34	Simulator vs Webster delay, x = 0.83	within 12 %	as expected	Pass
35	Simulator vs Webster delay, x = 0.67	within 12 %	as expected	Pass

7.2 Experimental set-up and metrics

Four controllers were compared: (1) the Webster fixed-time plan with green wave, recomputed per scenario; (2) the GA-tuned fixed-time plan (C = 73 s, arterial greens [43, 39, 43] s, offsets [0, 28, 68] s); (3) the hand-tuned fuzzy controller; and (4) the GA-tuned fuzzy controller. Each was run on 10 unseen seeds of each of the 6 scenarios. The primary metric is average delay per vehicle (s/veh). Secondary metrics are stops per vehicle, delay of vehicles entering from side streets (a fairness check) and vehicles left queued at the end. Differences against Webster are tested with a paired t-test on the 10 seeds (two-sided, 5 %, critical $|t| = 2.262$ for 9 degrees of freedom).

7.3 Convergence of the GA

Figure 7.1 shows the best fitness per generation for five independent runs of each GA. The fuzzy GA reached 29.27-30.14 s/veh on the training scenario (mean 29.49 ± 0.37). The hand-tuned controller scored 52.77 and Webster 37.93. The fixed-time GA reached 33.34-37.05 s/veh (mean 36.11 ± 1.56). Most of the gain of the fuzzy GA happens in the first 10 generations, and the runs end close to each other, so the result does not depend on a lucky seed. One GA run took about 112 s on 8 cores.

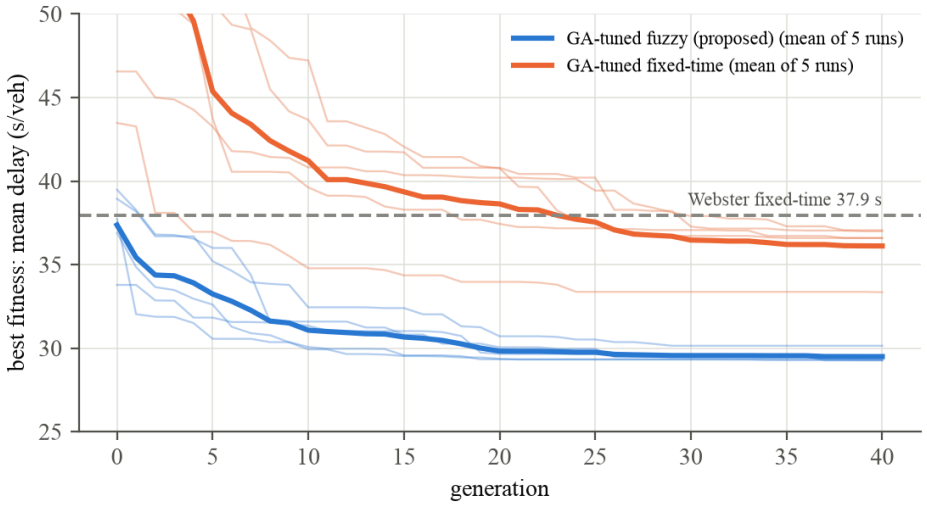


Figure 7.1: GA convergence: best fitness per generation (thin lines: individual runs; thick: mean of 5 runs).

Table 7.2: Best controller of each fuzzy GA run on training seeds and on the 10 test seeds.

Run	Training fitness (s/veh)	Test delay: Peak build-up and decay	Test delay: Peak	Test delay: Moderate
1	29.27	31.16	31.64	22.10
2	29.47	31.31	31.62	21.62
3	29.30	30.27	32.11	20.76
4	30.14	32.17	31.24	21.87
5	29.29	30.93	31.37	21.75
Mean ± sd	29.49 ± 0.37	31.17 ± 0.69	31.60 ± 0.33	21.62 ± 0.51

7.4 Tuned controller

Table 7.3 lists the controller before and after tuning. Figure 7.2 overlays the membership functions, and Figure 7.3 shows the resulting control surfaces.

Table 7.3: Controller parameters before and after GA tuning.

Parameter	Hand-tuned	GA-tuned
Green demand break points a, b, c (veh/lane)	2.00, 6.00, 12.00	0.00, 2.67, 3.86
Red queue break points a, b, c (veh/lane)	3.00, 8.00, 15.00	0.01, 3.00, 11.29
Switch threshold θ	0.50	0.63
Arterial min / max green (s)	10.0 / 60.0	13.5 / 55.0
Side-street min / max green (s)	7.0 / 40.0	8.2 / 33.4

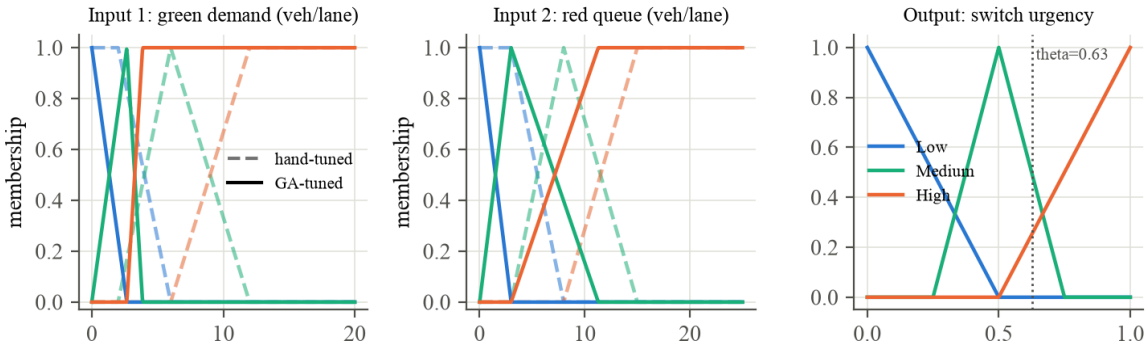


Figure 7.2: Membership functions: hand-tuned (dashed) and GA-tuned (solid); output sets with tuned threshold.

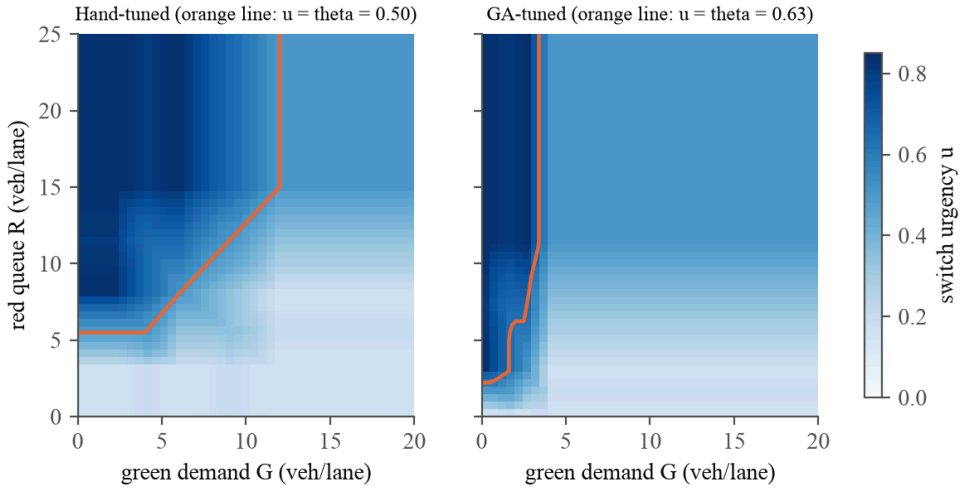


Figure 7.3: Control surfaces u(G, R). The green ends to the upper-left of the orange θ line.

The GA changed the controller in an interpretable way. The green-demand sets were compressed into the range 0-3.9 veh/lane: the tuned controller mainly asks whether the green approach is nearly empty (fewer than about 4 vehicles per lane queued or arriving within 7 s). This resembles the 'gap-out' logic of vehicle-actuated control, but it is graded by the red queue. The red-queue 'Medium' set was widened to 0.0-11.3 veh/lane, so the urgency rises gradually as cars accumulate on red. The resulting rule is roughly: end the green once the green approach has cleared, provided two or more vehicles per lane wait on red. Otherwise hold it, up to 55 s on the arterial and 33 s on a side street. The minimum arterial green rose from 10 s to 13.5 s, which protects arterial platoons from very short greens. The hand-tuned controller instead held the green while up to 12 veh/lane were still being served, which produced long cycles and wasted capacity.

7.5 Comparison with fixed-time plans

Table 7.4: Average delay (s/veh, mean ± sd over 10 test seeds). Bold = lowest. Last column: change in delay vs Webster (negative = less delay), paired t statistic and seeds won.

Scenario	Webster fixed-time	GA fixed-time	Fuzzy (hand-tuned)	GA-tuned fuzzy	Fuzzy vs Webster
Off-peak (light, steady)	**16.7 ± 0.3**	21.9 ± 0.3	30.6 ± 0.6	18.4 ± 0.4	+9.9 % (t = 11.6, 0/10)
Moderate (steady)	**21.4 ± 0.5**	26.3 ± 0.4	32.9 ± 0.5	22.1 ± 0.7	+3.1 % (t = 2.2, 3/10)
Peak (congested, steady)	42.4 ± 1.1	33.4 ± 1.3	54.2 ± 2.2	**31.6 ± 1.2**	-25.4 % (t = -31.1, 10/10)
Peak build-up and decay (training)	38.7 ± 2.5	33.9 ± 2.8	52.8 ± 3.8	**31.2 ± 2.3**	-19.6 % (t = -12.2, 10/10)
Tidal shift EB to WB	23.6 ± 1.4	27.4 ± 0.9	35.4 ± 1.6	**23.0 ± 0.8**	-2.2 % (t = -1.7, 6/10)
Side-street surge at J3	31.4 ± 4.9	42.8 ± 6.7	37.1 ± 1.5	**23.7 ± 1.1**	-24.6 % (t = -5.6, 10/10)

The results differ clearly between heavy, changing traffic and light, steady traffic. In the three congested or time-varying scenarios the GA-tuned fuzzy controller reduced average delay significantly and on every one of the 10 seeds: by 25.4 % in the steady peak (42.4 -> 31.6 s/veh), 19.6 % in the peak build-up used for training, and 24.6 % when the side streets of J3 surged. The surge scenario was never seen by the GA, so its large gain is evidence that the controller generalises. Under the tidal shift the change (+2.2 %, t = -1.65) is not statistically significant, and in moderate steady traffic the fuzzy controller was 3.1 % worse, also not significant ($|t| = 2.15 < 2.262$). In light off-peak traffic it was 9.9 % worse than Webster (16.7 vs 18.4 s/veh), and this difference is significant.

Two further comparisons separate the contribution of each technique. First, the **untuned** fuzzy controller was the worst controller in every scenario (52.8 s/veh in the peak build-up). The same nine rules became the best controller only after the GA placed the membership functions, so the evolutionary tuning is essential. Second, the GA applied to a **fixed-time** plan (same budget and fitness) improved on Webster only in the two heavy scenarios. The GA-tuned fuzzy controller beat the GA-tuned fixed-time plan in all six scenarios, on 10 of 10 seeds each (paired $|t| \geq 5.8$). The gain therefore comes from the combination: the GA alone cannot make a fixed plan adapt, and fuzzy rules alone are poorly calibrated.

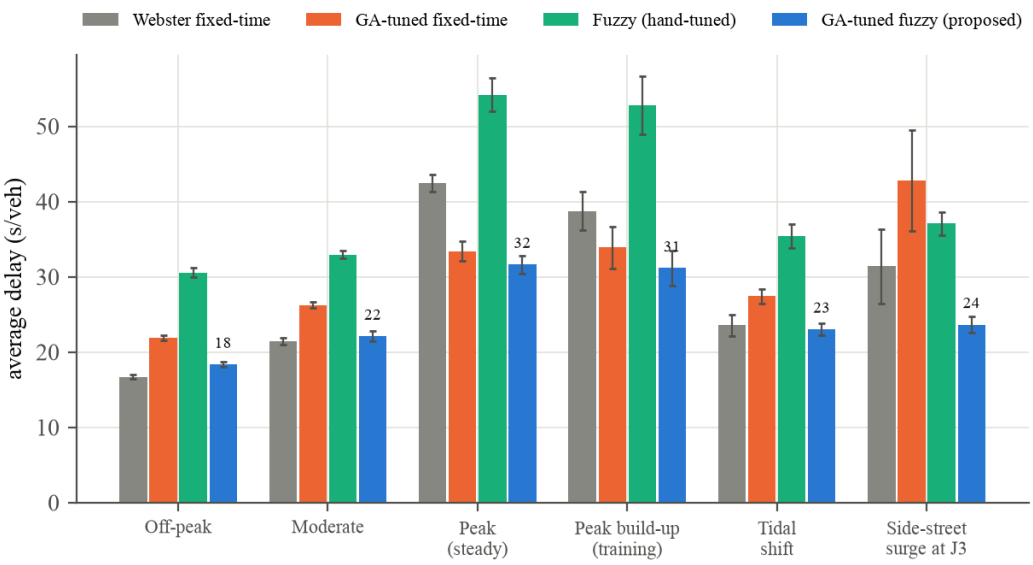


Figure 7.4: Average delay by scenario and controller (error bars: ±1 sd over seeds).

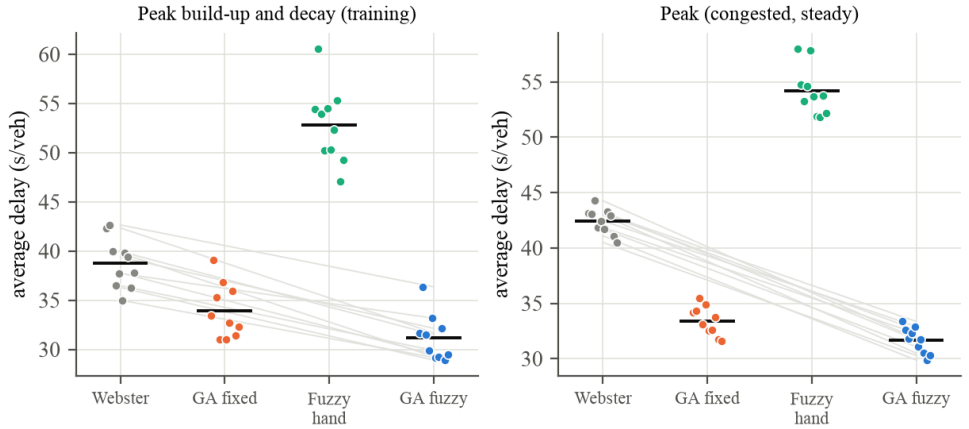


Figure 7.5: Per-seed delays in the two heaviest scenarios (grey lines join the same seed for Webster and GA fuzzy).

Table 7.5: Secondary metrics (means over 10 test seeds).

Scenario	Controller	Stops/veh	Side-street-origin delay (s)	Arterial-origin delay (s)	Left queued at end
Peak build-up and decay	Webster fixed-time	1.64	46.0	33.4	30
Peak build-up and decay	GA fixed-time	1.74	37.9	30.9	16
Peak build-up and decay	GA-tuned fuzzy	1.58	29.7	32.3	23
Peak	Webster fixed-time	1.57	46.7	39.3	70
Peak	GA fixed-time	1.79	36.4	31.3	46
Peak	GA-tuned fuzzy	1.61	30.7	32.3	51
Side-street surge at J3	Webster fixed-time	1.53	48.1	17.9	47
Side-street surge at J3	GA fixed-time	1.62	66.3	23.8	30
Side-street surge at J3	GA-tuned fuzzy	1.47	22.2	24.9	28

The fuzzy controller also reduced stops per vehicle in the congested scenarios (for example 1.58 against 1.64 for Webster in the peak build-up). It did not obtain its gains by starving the side streets: vehicles entering from side streets waited much less (30.7 s instead of 46.7 s in the steady peak, and 22.2 s instead of 48.1 s during the surge). The price is paid by vehicles entering on the arterial in lighter traffic. In the moderate scenario their delay rose from 15.9 s to 22.7 s, because Webster's green wave carries eastbound platoons through all three junctions, while the fuzzy junctions decide independently. This trade-off explains the moderate and off-peak results and is discussed in Chapter 8.

7.6 Behaviour over time and under increasing demand

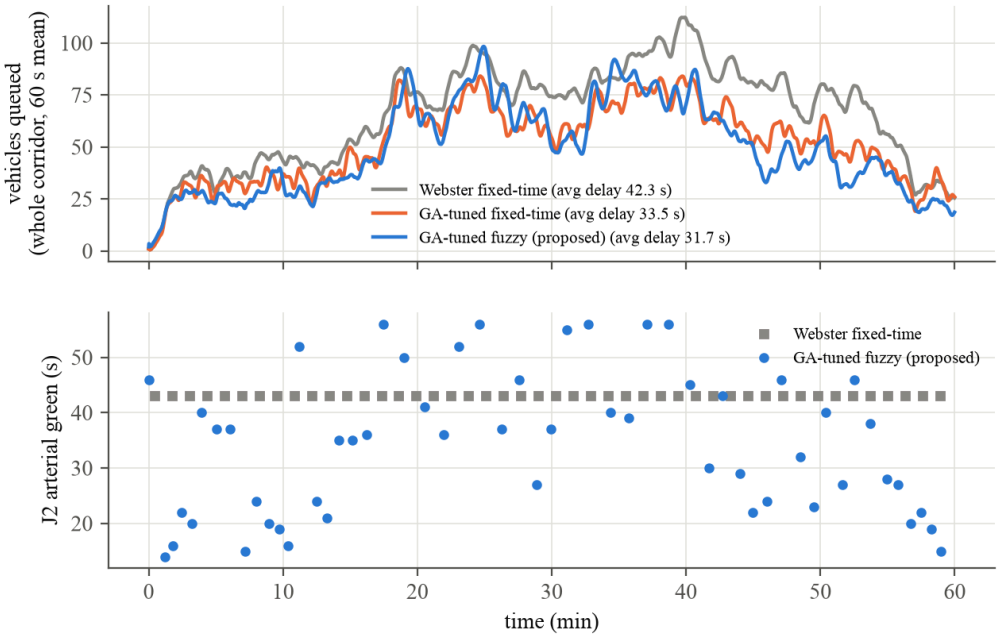


Figure 7.6: Peak build-up scenario, seed 1: vehicles queued in the corridor (top) and arterial green length at J2 (bottom).

In the peak build-up (seed 1) all controllers behave alike for the first 15 minutes. As demand rises, Webster's fixed 43 s arterial green at J2 cannot follow it, and after the peak its queue drains much more slowly. The fuzzy controller's arterial greens at J2 range from about 15 s in light periods to the 55 s maximum around the peak and fall again afterwards (lower panel). Over this hour the average delay was 42.3 s (Webster), 33.5 s (GA fixed-time) and 31.7 s (GA fuzzy).

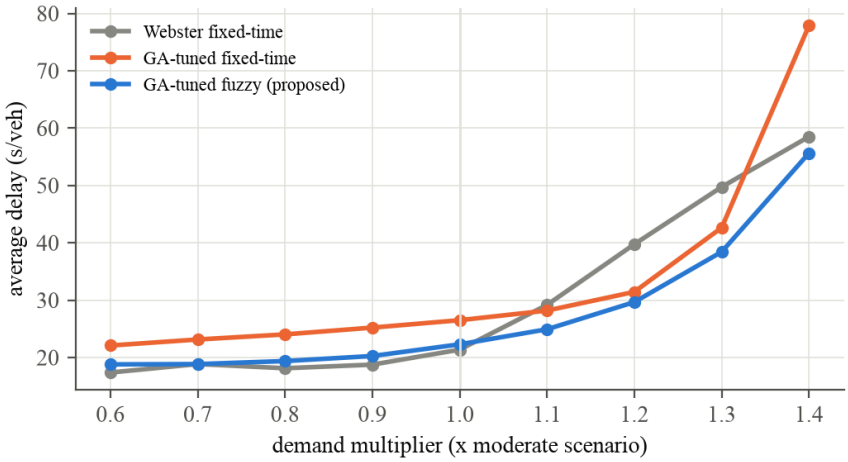


Figure 7.7: Average delay as all moderate-scenario flows are scaled from 0.6x to 1.4x (5 seeds per point).

Scaling every flow of the moderate scenario confirms the pattern. Below the design demand, Webster's plan (recomputed for each demand level) is slightly better. From 1.1x upwards the fuzzy controller is better, and the gap widens as the corridor approaches saturation: at 1.2x, 39.7 s against 29.7 s per vehicle. At 1.4x all controllers are oversaturated. The GA-tuned fixed plan, whose cycle was fitted to the training demand, then breaks down (77.9 s), whereas the fuzzy controller (55.7 s) stays below Webster (58.5 s).

7.7 Computational cost

One un-cached inference (fuzzification, 9 rules, aggregation on 101 points, centroid) takes 0.028 ms in pure Python, so a controller deciding once per second for each junction uses a negligible fraction of a processor. A one-hour simulation of the corridor takes about 0.1 s, and one complete GA run (3450 simulations) takes about 2 minutes on 8 cores.

8. Discussion

What the results show. A fuzzy controller tuned by a GA reduces corridor delay substantially when traffic is heavy or changes during the hour, which is the situation the problem statement describes (a congested corridor). In those conditions a fixed plan is wrong for most cycles. The fuzzy controller lengthens greens while platoons are still arriving, cuts them when an approach empties, and gives a surging side street the time it needs. Webster's formula also overestimates the best cycle near saturation (95 s in the steady peak), whereas the fuzzy controller settled at effective cycles of about 65-70 s.

Where it loses, and why. In light, steady traffic Webster's 40 s cycle with a green wave is hard to beat. The tuned controller produced effective cycles of about 60 s in the off-peak scenario. Its membership functions were fitted to the training hour, which is heavy, and they hold greens longer than light traffic needs. The second reason is coordination. The fuzzy junctions see upstream platoons only through the 7 s look-ahead and keep no common cycle

or offset, so on a lightly loaded arterial they break the green wave that a fixed plan maintains. Both reasons point to the training data and the controller inputs rather than to fuzzy logic itself. Section 8.2 tests the first one directly, and retraining on mixed demand removes the off-peak loss.

8.1 Effect of key parameters

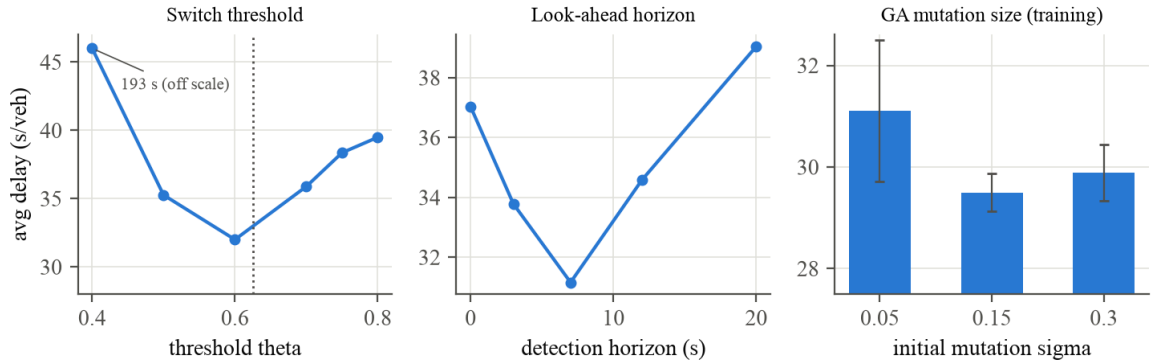


Figure 8.1: Sensitivity (peak build-up scenario): switch threshold θ, detection horizon, and initial GA mutation size.

Switch threshold θ. Delay is flat for θ between 0.5 and 0.7 (tuned value 0.63). Lowering θ to 0.4 is catastrophic (193 s/veh): the controller then ends greens as soon as the 'Medium' urgency rule fires, greens become very short, and the 4 s inter-green plus start-up lost time consume so much capacity that the corridor becomes oversaturated. This shows why tuning must respect the interaction between rules and lost time.

Detection horizon. Without look-ahead (0 s) the tuned controller's delay rises to 37.0 s/veh, against 31.2 s with the 7 s horizon it was tuned for. Information about vehicles about to arrive is therefore worth a large share of the gain: it is how a downstream junction learns that a platoon is coming. A longer horizon (20 s: 39.0 s/veh) is worse, because the controller then holds the green for vehicles that are still far away. This parameter was not part of the chromosome, so the controller is tuned to one horizon.

GA mutation size. The final training fitness for different initial mutation sizes was σ = 0.05: 31.10 ± 1.39 s (3 runs), σ = 0.15: 29.49 ± 0.37 s (5 runs), σ = 0.3: 29.89 ± 0.55 s (3 runs). The best value was σ = 0.15. A small step (0.05) makes the GA converge early to poorer controllers, because the population cannot move far enough from the initial individuals. The default schedule (0.15 shrinking to 0.03) balances exploration and fine tuning.

8.2 Extension: training on a mixture of demand levels

To test whether the off-peak weakness comes from the training data, the GA was run again (3 runs per design, same budget) with a fitness that averages one light, one moderate and one

peak-build-up hour (off-peak, moderate and peak build-up scenarios, seeds 1001-1003). The best controllers were then evaluated on the same 10 test seeds (Table 8.1).

Table 8.1: Average delay (s/veh) and change in delay vs Webster (negative = less delay) when the GA is trained on mixed demand (* = significant at 5 %, paired t-test).

Scenario	Webster	GA fuzzy (peak-trained)	GA fuzzy (mixed-trained)	GA fixed (mixed-trained)
Off-peak (light, steady)	16.7	18.4 (+9.9 %)	15.0 (-10.6 %*)	15.9 (-4.7 %)
Moderate (steady)	21.4	22.1 (+3.1 %)	20.6 (-3.8 %*)	20.3 (-5.3 %)
Peak (congested, steady)	42.4	31.6 (-25.4 %)	31.7 (-25.3 %*)	35.0 (-17.5 %)
Peak build-up and decay (training)	38.7	31.2 (-19.6 %)	30.5 (-21.2 %*)	35.6 (-8.0 %)
Tidal shift EB to WB	23.6	23.0 (-2.2 %)	22.1 (-6.2 %*)	22.2 (-5.6 %)
Side-street surge at J3	31.4	23.7 (-24.6 %)	23.1 (-26.3 %*)	36.4 (+15.9 %)

Training on mixed demand removed the weakness. The mixed-trained fuzzy controller reduced delay compared with Webster in 6 of 6 scenarios, 6 of them significantly: by 10.6 % off-peak (where the peak-trained controller was 9.9 % worse), 3.8 % in moderate traffic, 6.2 % under the tidal shift, and 21.2-26.3 % in the three heavy scenarios, so the heavy-traffic gains were kept. Its parameters explain the change. The red-queue 'Medium' set now starts at 0.6 veh/lane and peaks at 1.1, so a single waiting vehicle raises the urgency quickly in light traffic. The threshold rose to $\theta = 0.76$, so a green is ended only when the case for switching is clear. The arterial maximum green rose to 84 s for the heavy periods. The mixed-trained fixed-time plan ($C = 58$ s) became a good compromise in light traffic. However, it lost to the fuzzy controller in 5 of 6 scenarios (the exception is moderate traffic, 20.3 vs 20.6 s/veh) and it was 15.9 % worse than Webster during the side-street surge, which no single fixed plan can anticipate. The mixed-trained controller is therefore the recommended final design. The fitness function only had to describe the conditions the corridor actually experiences.

8.3 Limitations

- **Synthetic demand.** The scenarios use realistic design values but not field counts. The absolute delays are therefore model results. The relative ranking of controllers is the meaningful outcome.
- **Simplified traffic model.** Point queues, constant link travel times, no lane changing, no pedestrians and no heterogeneous Indian traffic (two- and three-wheelers). A microscopic simulator such as SUMO would give more realistic platoon dispersion.
- **Ideal detection.** The controller is assumed to see exact queues and arrivals. Real cameras and loops add counting errors.
- **Training demand.** The main experiment optimised on one demand pattern. Section 8.2 shows that this matters, and the mixed-trained controller was obtained from 3 GA runs rather than 5.

- **Fixed rule base and two phases.** Only the membership functions, threshold and green limits are evolved. Rule consequents, protected turn phases and pedestrian phases are not.
- **Mamdani approximation.** As noted in Section 7.1, max-aggregation makes the control surface only approximately monotone.

9. Real-World Application and SDG Relevance

Where it can be used. The controller needs only what adaptive signal systems already measure: the queue on each approach and the vehicles approaching from upstream. These come from video detection, inductive loops or radar. Its nine rules and eleven tuned parameters can run on an ordinary signal controller or a small embedded board. Because the rules are linguistic, traffic police and engineers can read, question and adjust them, which is harder with a neural-network controller. The GA tuning is done offline: a city would calibrate the simulator with its own counts for a corridor, run the GA, and load the tuned parameters into the junction controllers. The same approach applies to:

- Arterial corridors and bus routes with strongly varying or tidal demand (office and IT corridors, roads near markets, schools and stadiums).
- Junctions where a side street surges at known times (school dismissal, shift changes, events).
- Temporary signal control around construction zones, where fixed plans are rarely re-timed.
- Teaching and analysis: the simulator and GA can evaluate any candidate fixed-time or adaptive plan before field deployment.

SDG 11 - Sustainable Cities and Communities. Target 11.2 calls for safe, affordable, accessible and sustainable transport systems for all, and target 11.6 for reducing the adverse per-capita environmental impact of cities, including air quality [15]. Lower delay and fewer stops mean shorter journeys for buses and cars and less idling and stop-and-go driving, which are major sources of urban fuel use and emissions. Signal optimisation of this kind is a low-cost intervention: it reuses existing roads and signal hardware instead of building new capacity. Because the fitness function charges the full waiting time of side-street vehicles, the gains do not come from starving residents of minor roads, which supports the inclusive part of the goal.

10. Conclusion and Future Scope

This project formulated the design of a fuzzy traffic-signal controller for a congested three-junction corridor as an 11-variable optimisation problem. It solved the problem with a real-coded genetic algorithm and implemented the fuzzy inference system, the GA and the traffic

simulator from first principles in Python, without library functions. The simulator was validated against Webster's delay formula, and 35 unit tests check each component. On unseen random seeds the GA-tuned fuzzy controller reduced average vehicle delay against a Webster fixed-time plan with green-wave offsets by 25.4 % in steady peak traffic, 19.6 % in a peak that builds up and decays, and 24.6 % during a side-street surge. It also reduced stops and side-street waiting. It beat a GA-tuned fixed-time plan in all six scenarios, and the untuned fuzzy controller was the worst in all six, so neither technique alone explains the improvement. The peak-trained controller was worse than the coordinated fixed plan in light traffic. The analysis traced this to the training demand, and retraining the GA on a mix of light, moderate and peak hours produced a controller that reduced delay significantly in all six scenarios (3.8-26.3 %). Fuzzy rules give the controller a readable structure, and the genetic algorithm fits it to the traffic a corridor actually carries. The approach suits the congested, variable corridors of growing cities targeted by SDG 11, using detectors and signal hardware that already exist.

Future scope.

- Evolve the rule consequents together with the membership functions, as in [5], and compare with evolution strategies.
- Extend the mixed-demand training to a full day of time-of-day profiles, or use a multi-objective GA (delay, stops, emissions, maximum side-street wait) to obtain a set of trade-off controllers.
- Add explicit coordination inputs (offset to the upstream green) and more phases (protected turns, pedestrians).
- Calibrate with detector counts from a real Bengaluru corridor and validate in a microscopic simulator (SUMO) before field trials.
- Model mixed Indian traffic (two-wheelers, auto-rickshaws) and detector errors.

References

[1] L. A. Zadeh, "Fuzzy sets," Information and Control, vol. 8, no. 3, pp. 338-353, 1965.

[2] E. H. Mamdani and S. Assilian, "An experiment in linguistic synthesis with a fuzzy logic controller," International Journal of Man-Machine Studies, vol. 7, no. 1, pp. 1-13, 1975.

[3] C. P. Pappis and E. H. Mamdani, "A fuzzy logic controller for a traffic junction," IEEE Transactions on Systems, Man, and Cybernetics, vol. SMC-7, no. 10, pp. 707-717, Oct. 1977.

[4] M. B. Trabia, M. S. Kaseko, and M. Ande, "A two-stage fuzzy logic controller for traffic signals," Transportation Research Part C: Emerging Technologies, vol. 7, no. 6, pp. 353-367, Dec. 1999.

[5] A. Homaifar and E. McCormick, "Simultaneous design of membership functions and rule sets for fuzzy controllers using genetic algorithms," IEEE Transactions on Fuzzy Systems, vol. 3, no. 2, pp. 129-139, May 1995.

[6] F. V. Webster, "Traffic signal settings," Road Research Technical Paper No. 39, Her Majesty's Stationery Office, London, U.K., 1958.

[7] K. Deb and R. B. Agrawal, "Simulated binary crossover for continuous search space," Complex Systems, vol. 9, no. 2, pp. 115-148, 1995.

[8] J. H. Holland, Adaptation in Natural and Artificial Systems. Ann Arbor, MI, USA: University of Michigan Press, 1975.

[9] D. E. Goldberg, Genetic Algorithms in Search, Optimization and Machine Learning. Reading, MA, USA: Addison-Wesley, 1989.

[10] S. N. Sivanandam and S. N. Deepa, Principles of Soft Computing, 2nd ed. New Delhi, India: Wiley India, 2011.

[11] Transportation Research Board, Highway Capacity Manual, 6th ed. Washington, DC, USA: The National Academies Press, 2016.

[12] Indian Roads Congress, IRC:93-1985, Guidelines on Design and Installation of Road Traffic Signals. New Delhi, India: IRC, 1985.

[13] G. E. P. Box and M. E. Muller, "A note on the generation of random normal deviates," The Annals of Mathematical Statistics, vol. 29, no. 2, pp. 610-611, 1958.

[14] D. E. Knuth, The Art of Computer Programming, Vol. 2: Seminumerical Algorithms, 3rd ed. Reading, MA, USA: Addison-Wesley, 1997.

[15] United Nations General Assembly, "Transforming our world: the 2030 Agenda for Sustainable Development," Resolution A/RES/70/1, Oct. 2015.

[16] D. I. Robertson, "TRANSYT: a traffic network study tool," Road Research Laboratory, Crowthorne, U.K., Report LR 253, 1969.

[17] T. J. Ross, Fuzzy Logic with Engineering Applications, 3rd ed. Chichester, U.K.: Wiley, 2010.

[18] M. Matsumoto and T. Nishimura, "Mersenne twister: a 623-dimensionally equidistributed uniform pseudo-random number generator," ACM Transactions on Modeling and Computer Simulation, vol. 8, no. 1, pp. 3-30, 1998.

[19] Y.-C. Chiou and Y.-F. Huang, "Stepwise genetic fuzzy logic signal control under mixed traffic conditions," Journal of Advanced Transportation, vol. 47, no. 1, pp. 43-60, 2013, doi: 10.1002/atr.1205.

Appendix A: Sample Input and Output

Input (scenario 'peak', seed 1, GA-tuned fuzzy controller) and the program's output:

```
>>> import corridor as C, controllers as K, simulator as S
>>> sc = C.SCENARIOS["peak"]          # EB 1900, WB 1650 veh/h; side 350-500 veh/h
>>> fz = K.FuzzyController(best_genome)
>>> r = S.simulate(sc, fz, seed=1)
vehicles generated        : 6045
average delay (s/veh)     : 33.36    (Webster plan: 43.14)
average stops per vehicle : 1.59    (Webster plan: 1.58)
exited / queued / transit : 5954 / 39 / 52
max queue J1..J3 [EB,WB,NB,SB]: [[30, 25, 13, 10], [34, 31, 16, 17], [25, 22, 12,
13]]
```

Best genome found by the GA (11 genes):

```
0.000, 2.665, 1.198, 0.013, 2.989, 8.289, 0.626, 13.501, 55.035, 8.161, 33.421
```

Appendix B: Source Code Listing

Core modules (complete). The experiment scripts (optimise.py, evaluate.py), plotting (plots.py), the demo builder and the tests are in the submitted source folder.

src/rng.py

```
"""Random-number helpers built only on a uniform [0, 1) generator.

The only thing taken from Python is the Mersenne-Twister uniform generator
(random.Random.random). Gaussian draws (Box-Muller) and Poisson draws
(Knuth's multiplication method) are implemented here by hand so that the
soft-computing code does not depend on any library sampling routine.
"""
import math
import random


class RNG:
    def __init__(self, seed):
        self._r = random.Random(seed)
        self._spare = None  # second Box-Muller value

    def uniform(self):
        return self._r.random()

    def uniform_range(self, lo, hi):
        return lo + (hi - lo) * self._r.random()

    def randint(self, lo, hi):
        """Integer in [lo, hi] inclusive."""
        return lo + int(self._r.random() * (hi - lo + 1))

    def gauss(self):
        """Standard normal N(0, 1) via the Box-Muller transform."""
        if self._spare is not None:
            z, self._spare = self._spare, None
            return z
        u1 = self._r.random()
        while u1 <= 1e-12:
            u1 = self._r.random()
        u2 = self._r.random()
        radius = math.sqrt(-2.0 * math.log(u1))
        self._spare = radius * math.sin(2.0 * math.pi * u2)
        return radius * math.cos(2.0 * math.pi * u2)

    def poisson(self, lam):
        """Poisson(lam) count via Knuth's method (fine for lam < ~30)."""
        if lam <= 0.0:
            return 0
        limit = math.exp(-lam)
        k, p = 0, 1.0
        while True:
            p *= self._r.random()
            if p <= limit:
                return k
            k += 1
```

src/fuzzy.py

```
"""Mamdani fuzzy inference system written from first principles.

No fuzzy-logic library is used. The module provides
  * membership functions (left shoulder, triangle, right shoulder),
  * linguistic variables (fuzzification),
  * a Mamdani rule base with min (AND) / min (implication) /
    max (aggregation) and centroid defuzzification,
  * the traffic-signal "switch urgency" controller used in this project.
"""


# ---------------------------------------------------------------- membership functions

def left_shoulder(x, a, b):
    """1 for x <= a, falls linearly to 0 at x = b."""
    if x <= a:
        return 1.0
    if x >= b:
        return 0.0
    return (b - x) / (b - a)
```

```
def right_shoulder(x, a, b):
    """0 for x <= a, rises linearly to 1 at x = b."""
    if x <= a:
        return 0.0
    if x >= b:
        return 1.0
    return (x - a) / (b - a)


def triangle(x, a, b, c):
    """Triangle with feet a, c and peak b (a == b or b == c allowed)."""
    if x < a or x > c:
        return 0.0
    if x == b:
        return 1.0
    if x < b:
        return (x - a) / (b - a)
    return (c - x) / (c - b)


class LinguisticVariable:
    """A crisp universe [lo, hi] covered by named fuzzy sets."""

    def __init__(self, name, lo, hi, terms):
        self.name = name
        self.lo = lo
        self.hi = hi
        self.terms = terms  # dict: term name -> membership function of x

    def fuzzify(self, x):
        if x < self.lo:
            x = self.lo
        elif x > self.hi:
            x = self.hi
        return {t: mf(x) for t, mf in self.terms.items()}


def three_term_variable(name, lo, hi, a, b, c, labels=("Low", "Medium", "High")):
    """Low / Medium / High partition defined by three break points a < b < c.

    Low    : 1 up to a, 0 from b
    Medium : triangle (a, b, c)
    High   : 0 up to b, 1 from c
    The three memberships always sum to 1 (a strong fuzzy partition).
    """
    low, med, high = labels
    return LinguisticVariable(name, lo, hi, {
        low: lambda x, a=a, b=b: left_shoulder(x, a, b),
        med: lambda x, a=a, b=b, c=c: triangle(x, a, b, c),
        high: lambda x, b=b, c=c: right_shoulder(x, b, c),
    })


# ---------------------------------------------------------------- Mamdani inference

class MamdaniFIS:
    def __init__(self, inputs, output, rules, resolution=101):
        """
        inputs     : list of LinguisticVariable
        output     : LinguisticVariable (its universe is sampled for the centroid)
        rules      : list of (tuple of input term names, output term name)
        resolution : number of sample points used for centroid defuzzification
        """
        self.inputs = inputs
        self.output = output
        self.rules = rules
        step = (output.hi - output.lo) / (resolution - 1)
        self._ys = [output.lo + k * step for k in range(resolution)]
        self._out_mf = {t: [mf(y) for y in self._ys] for t, mf in output.terms.items()}

    def rule_strengths(self, *xs):
        """Firing strength of every rule (min of its antecedent memberships)."""
        mus = [var.fuzzify(x) for var, x in zip(self.inputs, xs)]
        strengths = []
        for antecedent, _ in self.rules:
            s = 1.0
            for mu, term in zip(mus, antecedent):
                if mu[term] < s:
                    s = mu[term]
            strengths.append(s)
        return strengths

    def aggregate(self, *xs):
        """Aggregated output fuzzy set sampled on the output universe."""
        clip = {t: 0.0 for t in self.output.terms}
        for s, (_, consequent) in zip(self.rule_strengths(*xs), self.rules):
```

```
            if s > clip[consequent]:
                clip[consequent] = s  # max over rules sharing a consequent
        agg = [0.0] * len(self._ys)
        for term, level in clip.items():
            if level <= 0.0:
                continue
            mf = self._out_mf[term]
            for k in range(len(agg)):
                v = mf[k] if mf[k] < level else level  # min implication (clipping)
                if v > agg[k]:
                    agg[k] = v                         # max aggregation
        return agg

    def infer(self, *xs):
        """Crisp output by the centroid (centre of gravity) method."""
        agg = self.aggregate(*xs)
        num = 0.0
        den = 0.0
        for y, mu in zip(self._ys, agg):
            num += y * mu
            den += mu
        if den == 0.0:
            return 0.5 * (self.output.lo + self.output.hi)
        return num / den


# ------------------------------------------------------------------ traffic-signal controller
FIS

# Rule base (expert knowledge, kept fixed; the EA tunes the membership functions).
# Output = urgency to end the current green.
#                 Red queue:  Low       Medium    High
SIGNAL_RULES = [
    (("Low", "Low"), "Low"),    (("Low", "Medium"), "High"),    (("Low", "High"), "High"),
    (("Medium", "Low"), "Low"), (("Medium", "Medium"), "Medium"), (("Medium", "High"),
"High"),
    (("High", "Low"), "Low"),   (("High", "Medium"), "Low"),    (("High", "High"), "Medium"),
]

GREEN_DEMAND_MAX = 20.0  # veh per lane
RED_QUEUE_MAX = 25.0     # veh per lane


def build_signal_fis(g_points, r_points):
    """g_points, r_points: (a, b, c) break points for the two input variables."""
    green = three_term_variable("green_demand", 0.0, GREEN_DEMAND_MAX, *g_points)
    red = three_term_variable("red_queue", 0.0, RED_QUEUE_MAX, *r_points)
    urgency = LinguisticVariable("switch_urgency", 0.0, 1.0, {
        "Low": lambda y: triangle(y, 0.0, 0.0, 0.5),
        "Medium": lambda y: triangle(y, 0.25, 0.5, 0.75),
        "High": lambda y: triangle(y, 0.5, 1.0, 1.0),
    })
    return MamdaniFIS([green, red], urgency, SIGNAL_RULES)
```

src/ga.py

```
"""Real-coded genetic algorithm written from scratch (no EA library).

Operators
  * initialisation : uniform random within bounds (+ optional seed individuals)
  * selection      : k-tournament
  * crossover      : simulated binary crossover (SBX), Deb & Agrawal 1995
  * mutation       : Gaussian (Box-Muller from rng.py), step size shrinks
                     linearly over the run
  * replacement    : generational with elitism
The GA minimises the fitness function.
"""
from rng import RNG


class RealCodedGA:
    def __init__(self, bounds, pop_size=30, generations=40, crossover_rate=0.9,
                 eta_c=15.0, mutation_rate=None, sigma_start=0.15, sigma_end=0.03,
                 tournament_size=3, elites=2, seed=0, seed_individuals=None):
        self.bounds = bounds
        self.n = len(bounds)
        self.pop_size = pop_size
        self.generations = generations
        self.pc = crossover_rate
        self.eta_c = eta_c
        self.pm = mutation_rate if mutation_rate is not None else 1.0 / self.n
        self.sigma_start = sigma_start
        self.sigma_end = sigma_end
        self.k = tournament_size
        self.elites = elites
        self.rng = RNG(seed)
```

```
    self.seed_individuals = seed_individuals or []

# ------------------------------------------------------------ helpers
def clamp(self, x):
    out = []
    for v, (lo, hi) in zip(x, self.bounds):
        if v < lo:
            v = lo
        elif v > hi:
            v = hi
        out.append(v)
    return out

def random_individual(self):
    return [self.rng.uniform_range(lo, hi) for lo, hi in self.bounds]

def tournament(self, pop, fit):
    best = self.rng.randint(0, len(pop) - 1)
    for _ in range(self.k - 1):
        c = self.rng.randint(0, len(pop) - 1)
        if fit[c] < fit[best]:
            best = c
    return pop[best]

def sbx(self, p1, p2):
    c1, c2 = list(p1), list(p2)
    if self.rng.uniform() > self.pc:
        return c1, c2
    for i in range(self.n):
        if self.rng.uniform() > 0.5:
            continue  # gene inherited unchanged
        u = self.rng.uniform()
        if u <= 0.5:
            beta = (2.0 * u) ** (1.0 / (self.eta_c + 1.0))
        else:
            beta = (1.0 / (2.0 * (1.0 - u))) ** (1.0 / (self.eta_c + 1.0))
        x1, x2 = p1[i], p2[i]
        c1[i] = 0.5 * ((1.0 + beta) * x1 + (1.0 - beta) * x2)
        c2[i] = 0.5 * ((1.0 - beta) * x1 + (1.0 + beta) * x2)
    return self.clamp(c1), self.clamp(c2)

def mutate(self, x, sigma_frac):
    y = list(x)
    for i, (lo, hi) in enumerate(self.bounds):
        if self.rng.uniform() < self.pm:
            y[i] += sigma_frac * (hi - lo) * self.rng.gauss()
    return self.clamp(y)

# ------------------------------------------------------------ main loop
def run(self, evaluate_batch, log=None):
    """evaluate_batch(list_of_genomes) -> list_of_fitness (lower is better)."""
    pop = [self.clamp(list(s)) for s in self.seed_individuals][: self.pop_size]
    while len(pop) < self.pop_size:
        pop.append(self.random_individual())
    fit = evaluate_batch(pop)
    evaluations = len(pop)
    history = []

    for gen in range(self.generations + 1):
        order = sorted(range(len(pop)), key=lambda i: fit[i])
        best_i = order[0]
        history.append({
            "generation": gen,
            "best": fit[best_i],
            "mean": sum(fit) / len(fit),
            "worst": fit[order[-1]],
            "best_genome": list(pop[best_i]),
            "evaluations": evaluations,
        })
        if log:
            log(history[-1])
        if gen == self.generations:
            break

        frac = gen / max(1, self.generations - 1)
        sigma = self.sigma_start + (self.sigma_end - self.sigma_start) * frac

        new_pop = [list(pop[i]) for i in order[: self.elites]]
        new_fit = [fit[i] for i in order[: self.elites]]
        children = []
        while len(new_pop) + len(children) < self.pop_size:
            p1 = self.tournament(pop, fit)
            p2 = self.tournament(pop, fit)
            c1, c2 = self.sbx(p1, p2)
            children.append(self.mutate(c1, sigma))
            if len(new_pop) + len(children) < self.pop_size:
                children.append(self.mutate(c2, sigma))
```

```
            child_fit = evaluate_batch(children)
            evaluations += len(children)
            pop = new_pop + children
            fit = new_fit + child_fit

        return history[-1]["best_genome"], history[-1]["best"], history
```

src/corridor.py

```
"""Geometry, signal timing constants and demand scenarios of the test corridor.

A stylised east-west urban arterial with three signalised junctions
J1, J2, J3 (J2 is the busiest cross street):

        N3            N2            N1      (side streets: 1 lane each way)
  W ----J1---350 m---J2---400 m---J3---- E  (arterial: 2 lanes each way)

Every junction runs two phases: phase 0 = arterial (EB + WB),
phase 1 = side street (NB + SB).
All values below are typical design values (see report, Chapter 4); the
demand is synthetic and is not field data from a real corridor.
"""

N_JUNCTIONS = 3
EB, WB, NB, SB = 0, 1, 2, 3
APPROACH_NAMES = ("EB", "WB", "NB", "SB")
PHASE_OF = (0, 0, 1, 1)                 # approach -> phase that serves it
PHASE_APPROACHES = ((EB, WB), (NB, SB))

LANES = (2, 2, 1, 1)                    # per approach
SAT_FLOW_PER_LANE = 1800.0              # veh/h/lane (base saturation flow)
SAT_RATE = tuple(l * SAT_FLOW_PER_LANE / 3600.0 for l in LANES)  # veh/s

LINK_LENGTH = (350.0, 400.0)            # m, J1-J2 and J2-J3
FREE_SPEED = 50.0 / 3.6                 # m/s (50 km/h)
LINK_TT = tuple(int(round(L / FREE_SPEED)) for L in LINK_LENGTH)  # 25 s, 29 s
JAM_SPACING = 7.0                       # m per queued car
LINK_STORAGE = tuple(int(2 * L / JAM_SPACING) for L in LINK_LENGTH)  # veh (2 lanes)

AMBER = 3
ALL_RED = 1
INTERGREEN = AMBER + ALL_RED            # s, no discharge
STARTUP_LOST = 2                        # s at the start of every green
LOST_PER_PHASE = INTERGREEN + STARTUP_LOST
LOST_PER_CYCLE = 2 * LOST_PER_PHASE     # 12 s
MIN_GREEN = 7

ARTERIAL_THROUGH = 0.85                 # rest turn off into side streets
SIDE_TO_EB = 0.15                       # side-street vehicles turning onto the arterial
SIDE_TO_WB = 0.15

DETECTION_HORIZON = 7                   # s of look-ahead (advance detector ~100 m upstream)
DURATION = 3600                         # s


class Scenario:
    """Entry demand streams; each has a base flow (veh/h) and a piecewise-linear
    multiplier profile [(t, m), ...] over the simulation horizon."""

    def __init__(self, name, label, eb, wb, side, profiles=None, duration=DURATION):
        self.name = name
        self.label = label
        self.duration = duration
        flat = [(0, 1.0), (duration, 1.0)]
        profiles = profiles or {}
        # stream key: (junction, approach)
        self.streams = [((0, EB), eb, profiles.get((0, EB), profiles.get("all", flat))),
                        ((2, WB), wb, profiles.get((2, WB), profiles.get("all", flat)))]
        for j, (nb, sb) in enumerate(side):
            self.streams.append(((j, NB), nb, profiles.get((j, NB), profiles.get("all",
flat))))
            self.streams.append(((j, SB), sb, profiles.get((j, SB), profiles.get("all",
flat))))

    @staticmethod
    def multiplier(profile, t):
        for (t0, m0), (t1, m1) in zip(profile, profile[1:]):
            if t0 <= t <= t1:
                if t1 == t0:
                    return m1
                return m0 + (m1 - m0) * (t - t0) / (t1 - t0)
        return profile[-1][1]

    def flow(self, stream, t):
        _, base, profile = stream
        return base * self.multiplier(profile, t)
```

```
    def mean_entry_flows(self):
        """Time-averaged flow (veh/h) of every entry stream."""
        out = {}
        for s in self.streams:
            total = sum(self.flow(s, t) for t in range(self.duration))
            out[s[0]] = total / self.duration
        return out


def approach_flows(entry):
    """Propagate mean entry flows through the corridor (turning fractions)
    and return flows[j][a] in veh/h arriving at every approach."""
    f = [[0.0] * 4 for _ in range(N_JUNCTIONS)]
    for j in range(N_JUNCTIONS):
        f[j][NB] = entry.get((j, NB), 0.0)
        f[j][SB] = entry.get((j, SB), 0.0)
    f[0][EB] = entry.get((0, EB), 0.0)
    for j in range(1, N_JUNCTIONS):
        f[j][EB] = ARTERIAL_THROUGH * f[j - 1][EB] + SIDE_TO_EB * (f[j - 1][NB] + f[j - 1]
[SB])
    f[N_JUNCTIONS - 1][WB] = entry.get((N_JUNCTIONS - 1, WB), 0.0)
    for j in range(N_JUNCTIONS - 2, -1, -1):
        f[j][WB] = ARTERIAL_THROUGH * f[j + 1][WB] + SIDE_TO_WB * (f[j + 1][NB] + f[j + 1]
[SB])
    return f


# ---------------------------------------------------------------------- scenarios

_MOD_SIDE = [(330, 300), (420, 390), (320, 280)]


def _scaled(side, k):
    return [(nb * k, sb * k) for nb, sb in side]


SCENARIOS = {
    "offpeak": Scenario("offpeak", "Off-peak (light, steady)",
                        900, 800, [(180, 160), (240, 220), (170, 150)]),
    "moderate": Scenario("moderate", "Moderate (steady)", 1500, 1300, _MOD_SIDE),
    "peak": Scenario("peak", "Peak (congested, steady)",
                     1900, 1650, [(400, 370), (500, 470), (390, 350)]),
    "peak_build": Scenario("peak_build", "Peak build-up and decay (training)", 1500, 1300,
_MOD_SIDE,
                           profiles={"all": [(0, 0.85), (1200, 1.3), (2400, 1.3), (3600,
0.85)]}),
    "tidal": Scenario("tidal", "Tidal shift EB to WB", 1500, 1300, _MOD_SIDE,
                      profiles={(0, EB): [(0, 1.3), (3600, 0.7)],
                                (2, WB): [(0, 0.7), (3600, 1.45)]}),
    "side_surge": Scenario("side_surge", "Side-street surge at J3", 1500, 1300, _MOD_SIDE,
                           profiles={(2, NB): [(0, 1), (1200, 1), (1260, 2.0), (2400, 2.0),
(2460, 1), (3600, 1)],
                                     (2, SB): [(0, 1), (1200, 1), (1260, 2.0), (2400, 2.0),
(2460, 1), (3600, 1)]}),
}

TRAINING_SCENARIO = "peak_build"
TRAINING_SEEDS = (1001, 1002, 1003)
TEST_SEEDS = tuple(range(1, 11))
```

src/simulator.py

```
"""Discrete-time (1 s) mesoscopic simulator of the three-junction corridor.

Model
  * Poisson vehicle arrivals at the 8 entry approaches (pre-generated from the
    seed, so every controller sees exactly the same vehicles: common random
    numbers).
  * Point (vertical) queues at every stop line, served FIFO at the saturation
    flow while the approach has effective green (green minus start-up lost time).
  * Vehicles leaving J1/J2/J3 either exit or travel to the next junction with
    a fixed free-flow link travel time.
  * Spill-back: a vehicle cannot enter a link whose storage (transit + queue)
    is full, which blocks its approach for that second.
  * Delay of a vehicle = total time spent waiting in stop-line queues.
    Vehicles still queued at the end are charged their waiting time so far.
"""
from collections import deque

import corridor as C
from rng import RNG


class Vehicle:
    __slots__ = ("t_gen", "delay", "stops", "u", "side_origin")
```

```
    def __init__(self, t_gen, u, side_origin):
        self.t_gen = t_gen
        self.delay = 0
        self.stops = 0
        self.u = u
        self.side_origin = side_origin


def generate_arrivals(scenario, seed):
    rng = RNG(seed)
    arrivals = {}
    for stream in scenario.streams:
        key = stream[0]
        side = key[1] in (C.NB, C.SB)
        lst = []
        for t in range(scenario.duration):
            k = rng.poisson(scenario.flow(stream, t) / 3600.0)
            for _ in range(k):
                u = tuple(rng.uniform() for _ in range(C.N_JUNCTIONS))
                lst.append(Vehicle(t, u, side))
        arrivals[key] = lst
    return arrivals


class CorridorSim:
    def __init__(self, scenario, seed, horizon=C.DETECTION_HORIZON):
        self.scenario = scenario
        self.horizon = horizon
        self.T = scenario.duration
        self.entry = generate_arrivals(scenario, seed)
        self.entry_ptr = {k: 0 for k in self.entry}
        self.vehicles = [v for lst in self.entry.values() for v in lst]
        nj = C.N_JUNCTIONS
        self.queues = [[deque() for _ in range(4)] for _ in range(nj)]
        self.credit = [[0.0] * 4 for _ in range(nj)]
        self.link_eb = [deque() for _ in range(nj - 1)]  # J(i) -> J(i+1): (arrival_time,
vehicle)
        self.link_wb = [deque() for _ in range(nj - 1)]  # J(i+1) -> J(i)
        self.exited = 0
        self.served = [[0] * 4 for _ in range(nj)]
        self.served_delay = [[0] * 4 for _ in range(nj)]
        self.max_queue = [[0] * 4 for _ in range(nj)]
        self.max_link_occupancy = [0] * (nj - 1)
        self.blocked_seconds = 0

    # ------------------------------------------------------------ sensors
    def queue_len(self, j, a):
        return len(self.queues[j][a])

    def approaching(self, j, a, t):
        """Vehicles that will join approach (j, a) within the next `horizon` s."""
        limit = t + self.horizon
        key = (j, a)
        if key in self.entry:
            lst, i, n = self.entry[key], self.entry_ptr[key], 0
            while i < len(lst) and lst[i].t_gen <= limit:
                n += 1
                i += 1
            return n
        link = self.link_eb[j - 1] if a == C.EB else self.link_wb[j]
        n = 0
        for arr, _ in link:
            if arr > limit:
                break
            n += 1
        return n

    # ------------------------------------------------------------ helpers
    def _destination(self, v, j, a):
        """Return ('eb', link), ('wb', link) or None (exit) for a discharged vehicle."""
        u = v.u[j]
        last = C.N_JUNCTIONS - 1
        if a == C.EB:
            go_eb, go_wb = u < C.ARTERIAL_THROUGH, False
        elif a == C.WB:
            go_eb, go_wb = False, u < C.ARTERIAL_THROUGH
        else:
            go_eb = u < C.SIDE_TO_EB
            go_wb = (not go_eb) and u < C.SIDE_TO_EB + C.SIDE_TO_WB
        if go_eb and j < last:
            return ("eb", j)
        if go_wb and j > 0:
            return ("wb", j - 1)
        return None

    def _link_full(self, dest):
```

```
    kind, i = dest
    if kind == "eb":
        occ = len(self.link_eb[i]) + len(self.queues[i + 1][C.EB])
    else:
        occ = len(self.link_wb[i]) + len(self.queues[i][C.WB])
    return occ >= C.LINK_STORAGE[i]

# ------------------------------------------------------------ main loop
def run(self, controller, record=False):
    controller.reset(C.N_JUNCTIONS)
    nj = C.N_JUNCTIONS
    trace = {"phase": [], "queues": [], "total_queued": []} if record else None

    for t in range(self.T):
        # 1. arrivals from outside the corridor
        for key, lst in self.entry.items():
            i = self.entry_ptr[key]
            q = self.queues[key[0]][key[1]]
            while i < len(lst) and lst[i].t_gen == t:
                q.append((lst[i], t))
                i += 1
            self.entry_ptr[key] = i
        # 2. arrivals from upstream links
        for i in range(nj - 1):
            dq = self.link_eb[i]
            while dq and dq[0][0] == t:
                self.queues[i + 1][C.EB].append((dq.popleft()[1], t))
            dq = self.link_wb[i]
            while dq and dq[0][0] == t:
                self.queues[i][C.WB].append((dq.popleft()[1], t))

        # 3. signals and discharge
        phases = []
        for j in range(nj):
            green, since = controller.signal(t, j, self)
            phases.append(green)
            for a in range(4):
                if green < 0 or C.PHASE_OF[a] != green or since < C.STARTUP_LOST:
                    self.credit[j][a] = 0.0
                    continue
                q = self.queues[j][a]
                self.credit[j][a] += C.SAT_RATE[a]
                while self.credit[j][a] >= 1.0 and q:
                    v, t_join = q[0]
                    dest = self._destination(v, j, a)
                    if dest is not None and self._link_full(dest):
                        self.blocked_seconds += 1
                        break
                    q.popleft()
                    self.credit[j][a] -= 1.0
                    wait = t - t_join
                    v.delay += wait
                    if wait > 0:
                        v.stops += 1
                    self.served[j][a] += 1
                    self.served_delay[j][a] += wait
                    if dest is None:
                        self.exited += 1
                    elif dest[0] == "eb":
                        self.link_eb[dest[1]].append((t + C.LINK_TT[dest[1]], v))
                    else:
                        self.link_wb[dest[1]].append((t + C.LINK_TT[dest[1]], v))
                if self.credit[j][a] > 1.0:  # no stored-up bursts (empty or blocked)
                    self.credit[j][a] = 1.0

        # 4. statistics
        for j in range(nj):
            for a in range(4):
                n = len(self.queues[j][a])
                if n > self.max_queue[j][a]:
                    self.max_queue[j][a] = n
        for i in range(nj - 1):
            occ = max(len(self.link_eb[i]) + len(self.queues[i + 1][C.EB]),
                      len(self.link_wb[i]) + len(self.queues[i][C.WB]))
            if occ > self.max_link_occupancy[i]:
                self.max_link_occupancy[i] = occ
        if record:
            qs = [[len(self.queues[j][a]) for a in range(4)] for j in range(nj)]
            trace["phase"].append(phases)
            trace["queues"].append(qs)
            trace["total_queued"].append(sum(sum(r) for r in qs))

    return self._results(trace)

def _results(self, trace):
    residual_queued = 0
    for j in range(C.N_JUNCTIONS):
```

```
            for a in range(4):
                for v, t_join in self.queues[j][a]:
                    v.delay += self.T - t_join
                    residual_queued += 1
        in_transit = sum(len(d) for d in self.link_eb) + sum(len(d) for d in self.link_wb)
        n = len(self.vehicles)
        side = [v for v in self.vehicles if v.side_origin]
        art = [v for v in self.vehicles if not v.side_origin]
        res = {
            "vehicles": n,
            "avg_delay": sum(v.delay for v in self.vehicles) / n if n else 0.0,
            "avg_stops": sum(v.stops for v in self.vehicles) / n if n else 0.0,
            "avg_delay_arterial_origin": sum(v.delay for v in art) / len(art) if art else 0.0,
            "avg_delay_side_origin": sum(v.delay for v in side) / len(side) if side else 0.0,
            "exited": self.exited,
            "residual_queued": residual_queued,
            "in_transit": in_transit,
            "max_queue": self.max_queue,
            "max_link_occupancy": self.max_link_occupancy,
            "served": self.served,
            "served_delay": self.served_delay,
            "blocked_seconds": self.blocked_seconds,
        }
        if trace is not None:
            res["trace"] = trace
        return res


def simulate(scenario, controller, seed, record=False, horizon=C.DETECTION_HORIZON):
    return CorridorSim(scenario, seed, horizon).run(controller, record)
```

src/controllers.py

```
"""Signal controllers: fixed-time plans (Webster / GA-tuned) and the fuzzy controller.

Every controller answers, once per second and per junction,
    signal(t, j, sim) -> (green_phase, seconds_since_green_start)
with green_phase = -1 during the inter-green (amber + all-red).
"""
import corridor as C
from fuzzy import build_signal_fis, GREEN_DEMAND_MAX, RED_QUEUE_MAX


# ============================================================ fixed-time plans

class FixedTimeController:
    """Common cycle, per-junction arterial green and offsets (coordinated plan).

    Cycle layout at junction j, with tau = (t - offset_j) mod C:
        [0, Ga)            arterial green
        [Ga, Ga+I)         inter-green
        [Ga+I, C-I)        side-street green
        [C-I, C)           inter-green
    """

    def __init__(self, cycle, art_green, offsets, name="Fixed-time"):
        self.cycle = cycle
        self.art_green = art_green
        self.offsets = offsets
        self.name = name

    def reset(self, n):
        pass

    def signal(self, t, j, sim):
        c, ga, ig = self.cycle, self.art_green[j], C.INTERGREEN
        tau = (t - self.offsets[j]) % c
        if tau < ga:
            return 0, tau
        if tau < ga + ig:
            return -1, 0
        if tau < c - ig:
            return 1, tau - ga - ig
        return -1, 0

    def describe(self):
        return {"cycle": self.cycle, "arterial_green": self.art_green,
                "side_green": [self.cycle - g - 2 * C.INTERGREEN for g in self.art_green],
                "offsets": self.offsets}


def split_plan(cycle, art_share, offsets, name):
    """Build a plan from a cycle, the arterial share of effective green and offsets."""
    cycle = int(round(cycle))
    eff = cycle - C.LOST_PER_CYCLE
    greens = []
```

```
    for s in art_share:
        g_art = int(round(s * eff)) + C.STARTUP_LOST
        g_side = cycle - 2 * C.INTERGREEN - g_art
        if g_side < C.MIN_GREEN:
            g_art = cycle - 2 * C.INTERGREEN - C.MIN_GREEN
        if g_art < C.MIN_GREEN:
            g_art = C.MIN_GREEN
        greens.append(g_art)
    return FixedTimeController(cycle, greens, [int(round(o)) % cycle for o in offsets], name)


def webster_plan(scenario, c_min=40, c_max=150):
    """Webster (1958) optimum cycle and equisaturation splits from the scenario's
    mean flows, with a common cycle and eastbound green-wave offsets."""
    flows = C.approach_flows(scenario.mean_entry_flows())
    L = C.LOST_PER_CYCLE
    cycles, shares, ys = [], [], []
    for j in range(C.N_JUNCTIONS):
        y_art = max(flows[j][C.EB] / (C.LANES[C.EB] * C.SAT_FLOW_PER_LANE),
                    flows[j][C.WB] / (C.LANES[C.WB] * C.SAT_FLOW_PER_LANE))
        y_side = max(flows[j][C.NB] / (C.LANES[C.NB] * C.SAT_FLOW_PER_LANE),
                     flows[j][C.SB] / (C.LANES[C.SB] * C.SAT_FLOW_PER_LANE))
        Y = y_art + y_side
        ys.append((y_art, y_side, Y))
        c0 = (1.5 * L + 5.0) / (1.0 - Y) if Y < 0.95 else c_max
        cycles.append(c0)
        shares.append(y_art / Y)
    cycle = min(c_max, max(c_min, max(cycles)))
    offsets = [0, C.LINK_TT[0], C.LINK_TT[0] + C.LINK_TT[1]]
    plan = split_plan(cycle, shares, offsets, "Webster fixed-time")
    plan.webster_info = {"flows_vph": flows, "flow_ratios": ys, "c0_per_junction": cycles}
    return plan


FIXED_BOUNDS = [(40.0, 150.0), (0.3, 0.85), (0.3, 0.85), (0.3, 0.85), (0.0, 1.0), (0.0, 1.0)]
FIXED_GENE_NAMES = ["cycle", "share_J1", "share_J2", "share_J3", "offset_J2", "offset_J3"]


def decode_fixed(genome, name="GA fixed-time"):
    cycle = int(round(genome[0]))
    return split_plan(cycle, genome[1:4], [0, genome[4] * cycle, genome[5] * cycle], name)


# ============================================================ fuzzy controller

FUZZY_BOUNDS = [
    (0.0, 6.0), (0.5, 7.0), (0.5, 7.0),     # green-demand break points a, b-a, c-b
    (0.0, 8.0), (0.5, 8.5), (0.5, 8.5),     # red-queue break points a, b-a, c-b
    (0.30, 0.80),                           # switch threshold theta (max urgency ~0.83)
    (7.0, 20.0), (25.0, 90.0),              # arterial min / max green (s)
    (7.0, 15.0), (15.0, 60.0),              # side-street min / max green (s)
]
FUZZY_GENE_NAMES = ["g_a", "g_b-a", "g_c-b", "r_a", "r_b-a", "r_c-b", "theta",
                    "gmin_art", "gmax_art", "gmin_side", "gmax_side"]

# Hand-designed starting point (engineering judgement, before any tuning).
HAND_FUZZY = [2.0, 4.0, 6.0, 3.0, 5.0, 7.0, 0.5, 10.0, 60.0, 7.0, 40.0]


def decode_fuzzy(genome):
    g_a = genome[0]
    g_b = g_a + genome[1]
    g_c = g_b + genome[2]
    r_a = genome[3]
    r_b = r_a + genome[4]
    r_c = r_b + genome[5]
    return {
        "g_points": (g_a, g_b, g_c),
        "r_points": (r_a, r_b, r_c),
        "theta": genome[6],
        "min_green": (genome[7], genome[9]),
        "max_green": (genome[8], genome[10]),
    }


class _JunctionState:
    __slots__ = ("phase", "in_green", "elapsed", "ig_left")

    def __init__(self):
        self.phase = 0
        self.in_green = True
        self.elapsed = 0
        self.ig_left = 0


class FuzzyController:
    """Per-junction Mamdani controller that decides, every second after the
```

```
minimum green, whether to end the current green.

Inputs (per lane, so 2-lane arterial and 1-lane side streets compare fairly)
  green_demand : max over green approaches of (queue + vehicles arriving
                 within the detection horizon) / lanes
  red_queue    : max over red approaches of queue / lanes
Output
  switch_urgency in [0, 1]; the green ends when urgency > theta,
  or when the maximum green is reached.
"""

def __init__(self, genome, name="Fuzzy"):
    self.genome = list(genome)
    self.params = decode_fuzzy(genome)
    self.fis = build_signal_fis(self.params["g_points"], self.params["r_points"])
    self.name = name
    self._cache = {}
    self.decisions = 0

def reset(self, n):
    self.state = [_JunctionState() for _ in range(n)]

def urgency(self, g, r):
    g = min(g, GREEN_DEMAND_MAX)
    r = min(r, RED_QUEUE_MAX)
    key = (int(round(g * 2)), int(round(r * 2)))  # inputs are multiples of 0.5
    u = self._cache.get(key)
    if u is None:
        u = self.fis.infer(key[0] / 2.0, key[1] / 2.0)
        self._cache[key] = u
    return u

def _inputs(self, t, j, phase, sim):
    g = 0.0
    for a in C.PHASE_APPROACHES[phase]:
        d = (sim.queue_len(j, a) + sim.approaching(j, a, t)) / C.LANES[a]
        if d > g:
            g = d
    r = 0.0
    for a in C.PHASE_APPROACHES[1 - phase]:
        d = sim.queue_len(j, a) / C.LANES[a]
        if d > r:
            r = d
    return g, r

def signal(self, t, j, sim):
    s = self.state[j]
    p = self.params
    if s.in_green:
        if s.elapsed >= p["min_green"][s.phase]:
            end = s.elapsed >= p["max_green"][s.phase]
            if not end:
                g, r = self._inputs(t, j, s.phase, sim)
                self.decisions += 1
                end = self.urgency(g, r) > p["theta"]
            if end:
                s.in_green = False
                s.ig_left = C.INTERGREEN
        if s.in_green:
            out = (s.phase, s.elapsed)
            s.elapsed += 1
            return out
    s.ig_left -= 1
    if s.ig_left == 0:
        s.phase = 1 - s.phase
        s.in_green = True
        s.elapsed = 0
    return -1, 0
```